49 B2

Shreya Gunjkar
1032240176
59 B2

MPJ ASSIGNMENT 2

Problem Statement :- Write a program in Java showing hierarchial inheritance with base class as Employee and derived classes as FulltimeEmployee and InternEmployee with methods DisplaySalary in base class and CalculateSalary in derived classes. Calculate salary methods will calculate as per increment given to fulltime and intern Employees. Fulltime employee - 50% hike, Inter employee - 25% hike. Display salary before and after hike

Objective :-

1. To study Inheritance in Java
2. To study why to use Inheritance
3. To study types of Inheritance

Theory :-

1. Inheritance in Java

Inheritance is a mechanism in Java by which one class (child class) acquires the properties and methods of another class (parent class)

→ The class that is inherited is called superclass.

→ The class that inherits is called subclass.

→ Inheritance is implemented using the extends keyword.

Eg-
```
class Employee {
    int salary = 20000;
}
class Manager extends Employee {
    int bonus = 5000;
}
```

2. Why to use Inheritance?
→ Inheritance is called because it provides:
(a) Code Reusability - Common code that can be written once and reused in child classes
(b) Time Saving - Reduces duplication of code
(c) Improves Maintainability - Changes made in parent class automatically reflect in child classes
(d) Supports Method Overriding - Enables runtime polymorphism.
(e) Represents Real-World Relationships
Ex: Employee → Manager

3. Types of Inheritance
(a) Single Inheritance - One child class inherits one parent class
class A { }
class B extends A { }
(b) Multilevel Inheritance - A class is derived from another derived class
class A { }
class B extends A { }
class C extends B { }
(c) Hierarchial Inheritance - Multiple child classes inherit from a single parent class
class A { }
class B extends A { }
class C extends A { }
(d) Hybrid Inheritance - Combination of more than one type of inheritance. Supported in Java using interfaces

Algorithm
1. Start
2. Create a base class Employee with data members salary and updatedSalary.
3. Define constructor to initialize salary values.
4. Create a derived class FullTimeEmployee and define calculateSalary() to apply 50% hike.

5. Create derived class InternEmployee, define calculateSalary() to apply 25% hike
6. In main() read salary for fulltimeEmployee and ~~create object~~ and calculate updated salary
7. Display salary before and applying hike
8. read salary and create object for InternEmployee and calculate updated salary.
9. Display salary before and after hike.
10. Stop

Conclusion :- Thus, we have successfully implemented usage of Inheritance in Java.

FAQs:

Q1.) Is multiple inheritance supported in Java? How is it achieved?
→ Not supported as it can cause ambuiguity.
It is achieved using interfaces as they contain abstract methods and java allows a class to implement multiple interfaces.

Eg.
```
interface A { void show(); }
interface B { void display(); }
class C implements A, B {
    public void show() {
        System.out.println("Show method");
    }
    public void display() {
        System.out.println("Display method");
    }
}
```

Q2.) What is Is-A relationship in Java?

→ An Is-A relationship represents inheritance. If a subclass inherits from a superclass, then the subclass Is-A type of the superclass.

Eg
```
class Animal {-}
class Dog extends Animal { }
```

Uses

- Enablems inheritance
- Supports method overidding
- Used for runtime polymorphism.

Q3.) Are constructors & instance initialization block inherited to subclass?

→ No, constructors and instance initialization blocks are not inherited to subclass as constructors initialize objects not inherited members and instance initialization blocks belong to a specific class. However, superclass constructor is called automatically using Super(). Instance initialization block executes before constructor, but still not inherited.

Eg
```
class A {
    A() {
        System.out.println("A constructor");
    }
}
class B extends A {
    B() {
        System.out.println("B constructor");
    }
}
```